Held

Build spec for Graph Hacks: Context for AI Agents

FalkorDB · WeMakeDevs · 15–18 October 2026

Hachemi Abdelaziz

Notes written 9 October 2026. Code starts 14 October 2026, 19:31 (UTC+1).

This file is a hackathon plan. It is not a DevLearnOps feature.
Do not deploy it on the production VPS. Do not update the company context file.

Contents

1 What this document is for **2**
1.1 What a win is . . . 2
1.2 Clock . . . 2

2 The official brief, and where Held meets it **3**

3 Architecture **3**
3.1 Request path . . . 4

4 Technologies **4**

5 Repository **5**

6 Graph model **5**
6.1 Labels . . . 6
6.2 Edges . . . 6
6.3 The three memories . . . 7

7 Queries **7**
7.1 `blockPath` . . . 7
7.2 `recordFinding` . . . 7
7.3 `proposeOrder` . . . 7
7.4 Concurrency check . . . 8

8 Agents and MCP tools **8**

9 Page and HTTP **8**

10 The check **8**

11 Build order **9**
11.1 Before 14 October, 19:31 . . . 9
11.2 After the window opens . . . 9

12 Shot list **9**

13 README, blog, submission **10**

14 Skills, plugins, and rules for the coding session **10**
14.1 First message for the coding chat 11

15 What will get the entry thrown out **11**

16 Source pages, fetched 9 October 2026 **11**

> **Not for treatment.** Held is a synthetic demo. The patients are fictional. The page must say that a clinician, not this software, decides what a real person takes. Do not load a real medication list. Do not give doses. Do not build drug discovery, a lab protocol, or anything that makes a substance.

1 What this document is for

On the first night of the hackathon, open this PDF and build only what it names. The product is one scene. The scene is the entry.

A ward agent records that ibuprofen must be held for Patient A, because a kidney finding and a public label say so. The chat is cleared. A pharmacy agent, in a new session, tries to restart ibuprofen. FalkorDB blocks the restart and returns the path: order, hold, finding, label, prescriber, admission, policy. Patient B has a headache and no hold. The same restart is allowed. Patient B is a different FalkorDB graph.

That is Held. An agent here is a function with tools. It reads the graph, takes one action, and writes the result back. It is not a personality and it is not a chatbot that happens to sit near a drawing of a graph.

1.1 What a win is

The event lists a $20,000 prize pool for every prize together. Track 2 pays a PS5 with GTA VI to each member of the one winning team. Track 1 pays an iPhone 18 Pro. Track 3 pays an M6 Mac Mini. A blog post pays AirPods 5. The top 50 submissions get a t-shirt. There is no cash grand prize above the tracks.

As of 9 October 2026 the public pages did not show a registrant count. A planning judgment, not a measurement: if the three beats below are live and obvious in the video, the chance at the PS5 is on the order of a few percent. If the demo looks like “these drugs interact,” the chance is about zero. The top-50 t-shirt is the result that finishing can realistically reach. This document does not raise that judgment. Adding frameworks does not raise it either.

Ship the public repo either way. A hiring person can open the repo and the video if the prize goes to someone else. The prize is one rank. The repo is the career artifact you control.

1.2 Clock

Times are Algeria (UTC+1). India Standard Time is UTC+5:30.

When	IST	What
Now until 14 Oct, 19:31	before 15 Oct, 00:01	Read, draw, run one hello query. Do not write the app.
14 Oct, 19:31	15 Oct, 00:01	Hacking starts. Create the repo.
18 Oct, 19:29	18 Oct, 23:59	Submissions close.

Re-read the rules that morning. The event page says the rules can change:
https://www.wemakedevs.org/hackathons/falkordb/rules

2 The official brief, and where Held meets it

The challenge page says the graph has to shape what the agent does, not sit on the side. If `proposeOrder` still returns the right allow or block while FalkorDB is stopped, the entry fails.

The site asks	Where it lives in Held
Follow connections across data	`blockPath` walks order, drug, hold, finding, label, person, admission, and policy.
Remember across sessions	The hold is still in the graph after the chat is cleared. `Attempt` nodes keep the history.
Share state between agents	The ward agent writes the hold. The pharmacy agent reads it. They do not pass a transcript. A `HANDOFF` edge records the hand.
Connect people, projects, and decisions	The prescriber and the pharmacist are people. The admission is the piece of work. The policy is the decision: a hold outlives the shift.
Show the path behind an answer	The page prints the rows of `blockPath`. The video shows those rows.

The brief's three verbs are the same scene. The pharmacy agent *acts* by allowing or blocking the restart. It *remembers* because the hold survives the empty chat. It *shows how* because the path is the reason on screen.

Track 2 on the event page also asks for episodic, semantic, and procedural memory, history, handoffs, concurrent reads and writes, and separate memory per tenant. Section 6 maps each of those words onto a node or an edge.

Track	What the judge sees	Prize
1. Agents that act	The restart is allowed or blocked because `blockPath` returned rows.	iPhone 18 Pro
2. Memory and coordination	New session, other agent, hold still in force, other patient isolated.	PS5 with GTA VI
3. Company brain	Who ordered, which admission, which policy, which label.	M6 Mac Mini

Enter all three tracks with this one repo. The video spends most of its time on the block in the new session. That is the PS5 scene. Tracks 1 and 3 are in the same path. They are not a second product.

3 Architecture

One Node process. One FalkorDB. Two graph names. No other database.

Piece	Role
`public/index.html`	Buttons and the path text. No framework.
`src/server.ts`	HTTP. Maps a patient id to a graph name.
`src/mcp.ts`	Same four tools, exposed as MCP.
`src/agents.ts`	Ward agent and pharmacy agent. They call tools only.
`src/graph.ts`	Every Cypher query and every write.
`src/seed.ts`	Synthetic patients. No hold on Patient A yet.
`src/check.ts`	The script that must pass.
FalkorDB	Graphs `held_a` and `held_b`.
Narrator, optional	One model call that reads rows and writes one sentence.

The page and the MCP server both call `src/graph.ts`. There is one decision. The decision is `proposeOrder`. A model may narrate the rows. A model must not choose allow or block. If the model is down, the block still happens.

> **Ceiling.** Two graphs and two parallel requests are the whole concurrency story. That is enough to show the track 2 line about concurrent reads and writes on a demo. A connection pool, a queue, and a load test are out. Upgrade only if a judge asks and Sunday morning is already shipped.

3.1 Request path

1. The page sends a pharmacy request for Patient A, with a session id and the drug id `ibuprofen`.
2. `src/server.ts` maps `patient_a` to graph `held_a`. Any other patient id returns 404 and creates no graph.
3. `proposeOrder` runs `blockPath`.
4. If the path has a row, the function writes an `Attempt` with result `blocked` and does not create an order. It writes a `HANDOFF` from the pharmacy agent to the ward agent.
5. If the path has no row, the function creates an `Order` with status `active` and an `Attempt` with result `allowed`.
6. The response is the decision plus the rows. The page prints them.

Stopping FalkorDB must make `GET /health` return 503 and `proposeOrder` throw. There is no in-memory fallback graph.

4 Technologies

Use the official client and the current docs. If this PDF and the docs disagree on a function name, the docs win. Read these on the first hour, before writing a call:

- https://docs.falkordb.com/commands/graph.query.html
- https://docs.falkordb.com/cypher/cypher-support
- https://docs.falkordb.com/getting-started/clients
- https://docs.falkordb.com/algorithms/
- the `falkordb` npm README
- the current TypeScript MCP quickstart, for the package name and the server boilerplate

Use	Why it is here
Docker image `falkordb/falkordb:latest`	The database. Copy ports from the current getting-started page. Do not guess 3000 or 6379 if that page names different ones.
Node 22 and TypeScript	You can explain it. One process serves HTTP, MCP, and the page.
npm package `falkordb` (`falkordb-ts`)	Official client. It issues `GRAPH.QUERY`. Parameterized queries only.
MCP TypeScript SDK	The hackathon lists MCP. Four tools, same functions as HTTP.
openFDA drug label API	Copy one short public sentence into the `Label` node during the event. Store the URL.

Do not use	Why
LangChain, LlamaIndex, a vector database	The decision must be Cypher. A second store lets the demo survive with FalkorDB turned off.
A Neo4j driver	FalkorDB is not Neo4j. No regex operator. No label expressions such as `(n:A|B)`.
GraphRAG SDK, an ORM, Graphiti-as-the-product	They hide the query you have to read aloud.
The DevLearnOps VPS, Kong, GHCR, Cloudflare, k3s	This product is a new repo on a new host.
A real patient's data	The rules forbid private personal data.

Graphiti is the production version of temporal memory, and FalkorDB documents it. Do not start there. After `npm run check` passes and the video is recorded, a stretch goal is to also store the ward note as a Graphiti episode. Skip that stretch if it threatens the deadline. The block stays in `src/graph.ts`.

Hybrid retrieval. The 2026 form of RAG is a search that finds a node, then a traversal that decides. Held does not need the search. The drug id is known. The traversal decides. Do not add an embedding to look impressive.

5 Repository

Create this on 14 October, after 19:31. Not before.

```
held/
  package.json
  tsconfig.json
  docker-compose.yml
  README.md
  src/server.ts
  src/mcp.ts
  src/agents.ts
  src/graph.ts
  src/seed.ts
  src/check.ts
  public/index.html
```

`package.json` scripts: `dev`, `seed`, `check`, `start`. License MIT. The README says the FalkorDB implementation was written during Graph Hacks.

6 Graph model

Two graphs. Same schema. No shared nodes. Isolation is the graph name passed to `GRAPH.QUERY`, not a `tenant_id` filter. A filter bug would leak a hold. A second graph cannot.

Graph	Patient	Hold at seed time
`held_a`	`patient_a`	none
`held_b`	`patient_b`	none

Store times as epoch milliseconds. FalkorDB's temporal arithmetic functions are unsupported, so do not do datetime math in Cypher.

6.1 Labels

Label	Properties	Seed
:Patient	id, name	A: patient_a, "Patient A". B: patient_b, "Patient B".
:Admission	id, title	adm_a "Night admission". adm_b "Night admission". This node is the project.
:Person	id, name, role	mina Mina Cole, prescriber. rafi Rafi Duran, pharmacist. Fictional. Same ids in both graphs.
:Agent	id	ward and pharmacy.
:Drug	id, name	ibuprofen, Ibuprofen.
:Order	id, status, at	On A only, at seed: one active order for ibuprofen, written by Mina. B has no order yet.
:Finding	id, kind, at	None on A at seed. The ward agent creates kidney_impairment. B's seed includes headache and no hold.
:Hold	id, valid_at, invalid_at	Created only by the ward agent. invalid_at stays null. Null means the hold is in force.
:Label	id, url, sentence	ibuprofen_renal. URL and sentence copied from openFDA on day 1.
:Policy	id, text	hold_survives_shift. Text: "A hold stays in force across sessions until a clinician clears it."
:Attempt	id, at, session	Created by proposeOrder. This is the history.

6.2 Edges

Edge	Meaning
(Patient)-[:HAS]->(Admission)	The case belongs to the patient.
(Person)-[:OWNS]->(Admission)	Mina owns the admission.
(Person)-[:ORDERED]->(Order)	Mina wrote the original order on A.
(Order)-[:OF]->(Drug)	The order is for ibuprofen.
(Order)-[:DURING]->(Admission)	The order is part of the case.
(Finding)-[:ON]->(Admission)	The finding is about this case.
(Agent)-[:RECORDED]->(Finding)	The ward agent wrote it.
(Hold)-[:BLOCKS]->(Drug)	Procedural memory: do not restart this drug.
(Hold)-[:BECAUSE]->(Finding)	Why the hold exists.
(Hold)-[:CITES]->(Label)	The public source.
(Hold)-[:UNDER]->(Policy)	The decision the hold carries out.
(Agent)-[:WROTE]->(Hold)	The ward agent wrote the hold.
(Attempt)-[:TRIED]->(Drug)	Properties: result, at. Result is blocked or allowed.
(Agent)-[:HANDOFF]->(Agent)	Properties: at, reason, attempt_id.

6.3 The three memories

Word on the track page	Node or edge
Episodic	`:Attempt` and `:Finding`, with `at`.
Semantic	`:Drug`, `:Label`, `:Person`, `:Admission`, `:Policy`.
Procedural	`:Hold` whose `invalid_at` is null. Meaning: do not restart.
Persistent history	Attempts remain after the page session id changes.
Shared state	The pharmacy agent reads the hold the ward agent wrote. `HANDOFF` is the evidence in Browser.
Separate tenant	`held_a` and `held_b`.

Headache does not create a hold. The only finding kind that creates a hold is `kidney_impairment`, and only because the seed linked that kind to the label node. The agent does not diagnose.

7 Queries

Drafts. Run each one in FalkorDB Browser before `src/graph.ts` calls it. `WHERE` after `OPTIONAL MATCH` is easy to get wrong, so the block query counts the hold instead.

7.1 `blockPath`

```
MATCH (d:Drug {id: $drug})<-[:BLOCKS]-(h:Hold)
WHERE h.invalid_at IS NULL
MATCH (h)-[:BECAUSE]->(f:Finding)
MATCH (h)-[:CITES]->(l:Label)
MATCH (h)-[:UNDER]->(p:Policy)
MATCH (f)-[:ON]->(adm:Admission)<-[:OWNS]-(prescriber:Person {role: 'prescriber'})
RETURN d.name AS drug,
       h.id AS hold,
       f.kind AS finding,
       l.sentence AS label,
       l.url AS url,
       p.text AS policy,
       prescriber.name AS prescriber,
       adm.id AS admission
```

No rows means allow. One row means block. The page prints every returned field. That printout is “show the path.”

7.2 `recordFinding`

For `kind = kidney_impairment` on `held_a`, in one query batch:

1. Create the finding. Link the ward agent to it with `RECORDED`. Link the finding to the admission with `ON`.
2. Create the hold. Set `valid_at` to now. Leave `invalid_at` null.
3. Add `BLOCKS`, `BECAUSE`, `CITES`, and `UNDER`. Link the ward agent to the hold with `WROTE`.
4. Add a `HANDOFF` from the ward agent to the pharmacy agent. Reason: “Hold written. Pharmacy must read it before a restart.”

For `headache`, create the finding only. Do not create a hold. Unknown `kind`: write nothing and return an error.

7.3 `proposeOrder`

1. Run `blockPath`.

2. Create an `Attempt` and `(attempt)-[:TRIED {result, at}]->(drug)`.
3. On `blocked`, do not create an `Order`.
4. On `allowed`, create an active order. The pharmacist `ORDERED` it, the order is `OF` the drug, and it happened `DURING` the admission.
5. Create `(pharmacy)-[:HANDOFF]->(ward)` with the attempt id.

The drug id comes from the button, which only sends `ibuprofen`. Reject any other drug id with 400 and write nothing.

7.4 Concurrency check

After the hold exists, fire two `proposeOrder` calls on `held_a` without waiting between them. Both results must be `blocked`. Both must have written an `Attempt`. Neither may create an `Order`. Put this in `src/check.ts`.

8 Agents and MCP tools

`src/agents.ts` has two functions.

Agent	Tool	Write
Ward	`record_finding`	Finding, and a hold when the kind is `kidney_impairment`.
Pharmacy	`propose_order`	Attempt, handoff, and an order only when the path is empty.
Either	`why`	Read only. Returns `blockPath`.
Either	`health`	Read only. Runs `RETURN 1`.

The MCP server exposes those four tools. Tool descriptions say they read or write FalkorDB. The HTTP routes call the same functions, so the demo does not depend on an MCP client being installed on the judge's laptop. The README shows one `tools/list` dump so the MCP surface is visible.

If the official SDK's boilerplate takes more than 90 minutes, ship HTTP first and add MCP only after `npm run check` is green. Do not invent a second decision path to save time.

A union `'allow' | 'block'` needs a `never` check in the default branch, so a third state fails the TypeScript build.

9 Page and HTTP

Call	Body	Result
POST patient A, ward	kind `kidney_impairment`	The new hold id.
POST patient A, pharmacy	session id and drug `ibuprofen`	`decision` and `rows`.
POST patient B, pharmacy	same body	`allowed`.
GET `/health`	none	200, or 503 if the query fails.

One `public/index.html`. Controls: Patient A, Patient B, "Record kidney finding," "Restart ibuprofen," and a `pre` that prints the JSON. A banner at the top: "Synthetic demo. Not for treatment." No login. No second page. No graph drawing library. The JSON path is the picture. Add a drawing only after the video is recorded.

10 The check

`npm run check` is one Node script. No test framework. Exit code 0 only when every step holds.

1. Delete graphs `held_a` and `held_b` if they exist, then seed them.
2. `proposeOrder('held_a', ibuprofen)` returns `allowed`. Seed has an order already. This call is a second allow, before any hold.
3. `recordFinding('held_a', kidney_impairment)` creates a hold with `invalid_at` null.
4. `proposeOrder('held_a', ibuprofen)` returns `blocked`. The rows include Mina Cole, the policy text, and the label sentence. No new `Order` is created by this call.
5. Two parallel proposes on `held_a` both return `blocked`.
6. `proposeOrder('held_b', ibuprofen)` returns `allowed`.
7. A query for a hold on `held_b` returns no rows.

If step 4 passes while Docker is stopped, delete the fallback. The last manual step, written in the README, is: stop FalkorDB, `GET /health` returns 503, start it, the block works again.

11 Build order

Each step ends with something you can see. Do not start the next step early.

11.1 Before 14 October, 19:31

1. Register on the hackathon page. Join the FalkorDB Discord.
2. `docker pull falkordb/falkordb:latest`. Start it with the ports from the current docs. Open FalkorDB Browser.
3. Type one query yourself: two nodes, one relationship, then `MATCH` them.
4. Draw the labels from Section 6 on paper.
5. Open openFDA and find the ibuprofen label. Copy one sentence that mentions kidney or renal impairment, plus the URL, into a note. If that sentence is not on the current label, pick the warning sentence that is, and keep the finding kind matched to that sentence. Do not invent the sentence.
6. Read the shot list aloud. Time it.
7. Choose a new host for the live URL. Not the DevLearnOps machine.

Do not create the GitHub repo. Do not write TypeScript. Do not scaffold.

11.2 After the window opens

1. Create the public repo `held`.
2. Compose for FalkorDB only. Browser opens.
3. Write `npm run check` so it fails on a missing graph.
4. Seed and queries until the check passes. Paste the final Cypher into the README the same day.
5. HTTP routes from the table above.
6. The one page. Click Patient A, record the finding, restart, see `blocked` and the rows. Switch to Patient B, restart, see `allowed`.
7. MCP `tools/list` showing the same four tools.
8. Parallel block check from the check script.
9. Deploy the same Compose stack. `GET /health` on the live URL.
10. Record the video. Write the blog. Submit tracks 1, 2, and 3, and the blog URL.

If Sunday afternoon comes and deploy is shaky, stop features. A live block beats a Graphiti episode.

12 Shot list

Say these words while the screen shows the action.

1. “Patient A is already on ibuprofen. Nothing is held.” Show the allow, or the seeded active order, and an empty hold query.
2. “The ward agent records the kidney finding. The hold cites the public label and the policy that a hold survives the shift.” Click the ward button. Show the new edges in Browser.
3. “New session. Empty chat. The pharmacy agent tries to restart ibuprofen.” Click restart. The page shows `blocked` and the rows: Mina Cole, the finding, the label sentence, the policy.
4. “Patient B is a different graph. A headache does not copy A’s hold.” Switch patient. Click restart. The page shows `allowed`.
5. “Stop the database and this button cannot decide.” Stop FalkorDB if the recording setup allows it. Otherwise cut in the README kill test as a still.

Keep it under three minutes. Cut any sentence that is not one of those five beats.

13 README, blog, submission

The README has these sections, in this order:

1. The three beats, in plain sentences.
2. Docker and npm commands copied from what you ran.
3. The graph model, updated if a property changed.
4. The final Cypher, pasted from `src/graph.ts`.
5. The live URL, or the local commands and the video file name.
6. Tracks entered: 1, 2, and 3.
7. AI disclosure: name the tools, and state that `proposeOrder` is deterministic Cypher.
8. The kill test.
9. The not-for-treatment line.

Blog title: “The next shift was not allowed to restart ibuprofen.” Publish on a site you already control. Cover the empty chat, the hold edge, Patient B, the query, and the 503. Two hours maximum. Submit the URL for the AirPods side prize.

Submission packet: public repo, live URL, video under three minutes, blog URL, all three tracks named, sent before 18 October 2026, 19:29 (UTC+1).

14 Skills, plugins, and rules for the coding session

Load a skill only when that step starts. Do not open the skill catalog. Do not load marketing, ISO, payments, Strix, or Lemon Squeezy. This product does not take money.

When	What to open
Any code	Ponytail, already on in `.cursor/rules/ponytail.mdc`. The smallest code that makes `npm run check` pass.
Before a FalkorDB call	The current page on `https://docs.falkordb.com` for that call.
Docker will not start	The `docker-development` skill, then return to this PDF.
The page exists	Prove the beats in the browser. Click ward, pharmacy, and the patient switch. A screenshot is not proof. Use the Cursor browser tools for that proof only.
Before you say it is done	The `verification-loop` skill: the check output, the live URL, and the video file.
README and the blog	Technical writing. Short sentences. Paste the real queries.
A TypeScript union	The workspace rule for an exhaustive switch. `allow` and `block` only.

Plugins: the Cursor browser is for the proof. Do not build Held as a Cursor chat memory, a Gmail demo, or an X demo. Do not call the Upwork plugin.

Rules that bind the coding session:

1. Hackathon rules at the URL in the clock section. FalkorDB is load-bearing. New application code starts after 14 October 2026, 19:31 (UTC+1). AI use is disclosed. You can explain the model without this PDF.
2. Ponytail. No new dependency if the official `falkordb` package and the MCP SDK already cover it.
3. Imports at the top of the file.
4. Company context: read it only to avoid the production cluster. Do not edit the company context file. This hackathon is not the monthly rock.
5. Synthetic data only. The not-for-treatment banner stays.
6. Do not commit secrets. No API keys in the repo. The narrator key, if any, stays in an uncommitted env file.

14.1 First message for the coding chat

Paste this after 19:31 on 14 October:

> Build Held from `hackathon/held/held.pdf`. The lab spec in `hackathon/second-shift/` is retired. Do not touch DevLearnOps production. Start at the failing `npm run check`, then the queries in the PDF. The pharmacy agent blocks only when `blockPath` returns a row. Patient B must still allow ibuprofen. Stop when the check passes, the three beats work in the browser, and the live URL returns 503 with FalkorDB down.

15 What will get the entry thrown out

- The product still decides after FalkorDB is stopped.
- The graph is a picture of data that actually lives in Postgres, a JSON file, or the chat log.
- Application code was committed before 14 October 2026, 19:31 (UTC+1). Notes and this PDF are allowed. The repo is not.
- Private, login-gated, or real patient data.
- You cannot explain the labels, the edges, and why the restart was blocked.
- The project is only generated output, with no check you ran.
- A second product stuffed in to chase the iPhone and the Mac Mini.
- A medical claim. Held blocks a synthetic order in a demo. It does not treat anyone.

16 Source pages, fetched 9 October 2026

- https://www.wemakedevs.org/hackathons/falkordb
- https://www.wemakedevs.org/hackathons/falkordb/rules
- https://www.wemakedevs.org/hackathons/falkordb/resources
- https://docs.falkordb.com/agentic-memory/graphiti
- https://www.falkordb.com/blog/graphiti-falkordb-multi-agent-performance/
- https://www.who.int/publications/i/item/WHO-UHC-SDS-2019.9
- https://open.fda.gov/

The WHO page is why the scene is a shift change. It is not permission to build a clinical system. openFDA is the place to copy the one label sentence. SIDEKICK and other full drug graphs are out of scope for these four days.